SRE CASE STUDY

AZURE ARC HYBRID CLOUD LAB

A Proof-of-Value project demonstrating hybrid cloud orchestration, observability, FinOps, and Infrastructure-as-Code practices using K3s Kubernetes and Microsoft Azure Arc.

Author: Bartosz Suszko

Company: IT Solutions Bartosz Suszko, Olsztyn, Poland

Date: May 2026

GitHub: github.com/buth11/sre-homelab-azure-arc

Stack: K3s v1.35.4 | Azure Arc | Prometheus | Grafana | Terraform | KQL

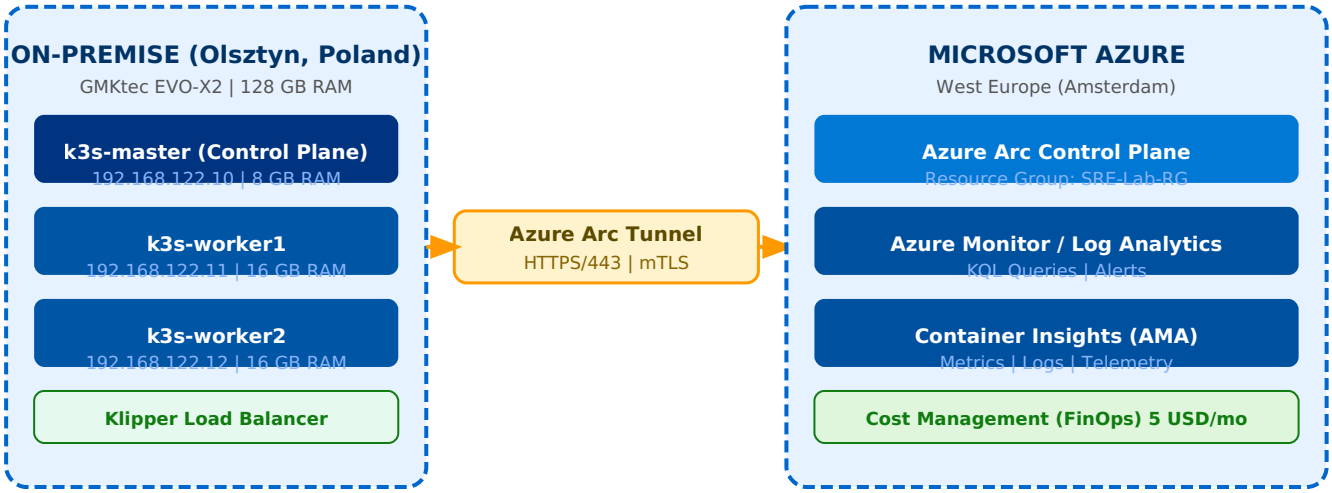
Table of Contents

Chapter 1: Business Context and Hybrid Architecture

This project demonstrates the design, deployment, and operation of a production-grade hybrid environment connecting on-premises infrastructure with Microsoft Azure. Built on dedicated hardware (GMKtec EVO-X2, 128 GB RAM), it uses the same technology stack found in commercial enterprise SaaS deployments.

The goal is to demonstrate SRE competencies in: hybrid infrastructure management, observability, automation, and cost control — all required for maintaining a SaaS platform for enterprise customers.

Architecture Diagram



Technology Stack

Layer	Technology	Purpose
Hypervisor	KVM on Fedora 43	Node virtualization
Operating System	Ubuntu Server 24.04 LTS	All K3s nodes
Kubernetes	K3s v1.35.4+k3s1	Lightweight K8s distribution
Cloud Bridge	Azure Arc (agent v1.33.0)	Hybrid control plane
Cloud Monitoring	Azure Monitor + Container Insights	Telemetry and KQL analytics
Local Monitoring	Prometheus + Grafana v3.21	Open-source observability stack
IaC	Terraform ~3.100	Infrastructure as Code
Package Manager	Helm v3.21.0	K8s application deployment
Load Balancer	K3s Klipper LB	Layer 4 round-robin routing
FinOps	Azure Cost Management	Budget guardrail: 5 USD/month

Chapter 2: On-Premise Infrastructure — K3s Cluster

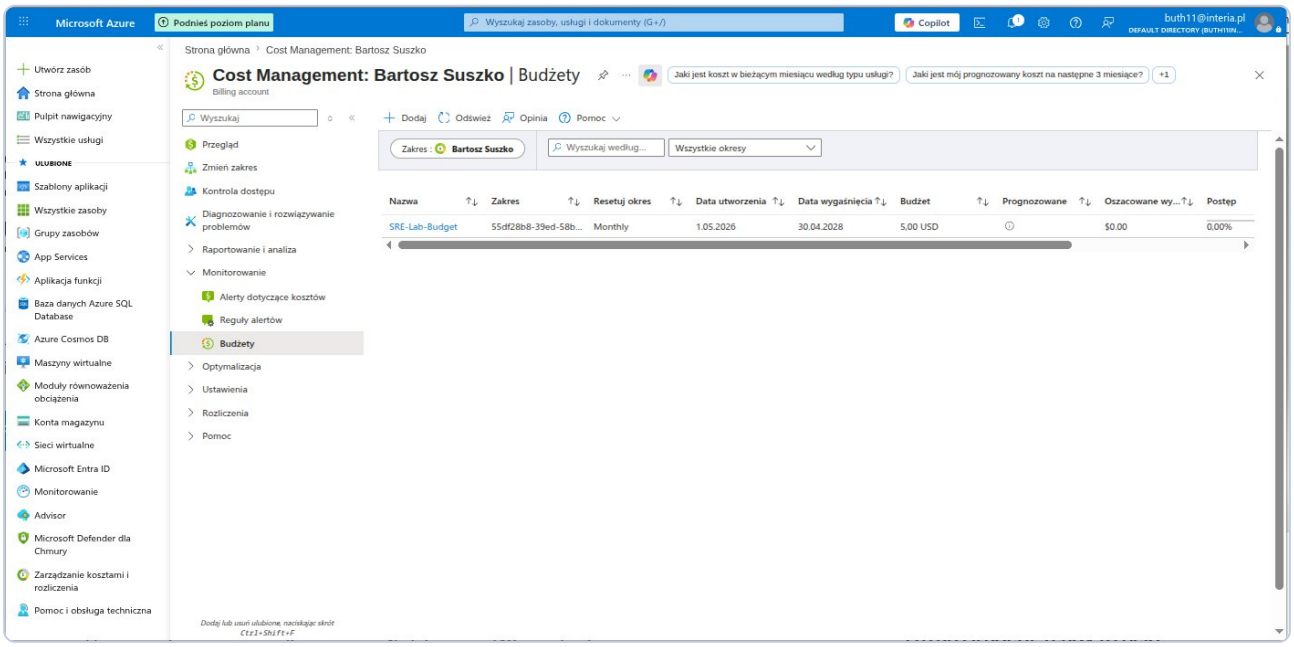
The K3s cluster consists of three virtual machines running on GMKtec EVO-X2 (128 GB RAM). Each node has a static IP address in the 192.168.122.0/24 subnet and runs Ubuntu Server 24.04 LTS.

Node	Role	IP	CPU	RAM	Disk
k3s-master	Control Plane	192.168.122.10	4 vCPU	8 GB	40 GB
k3s-worker1	Worker	192.168.122.11	4 vCPU	16 GB	60 GB
k3s-worker2	Worker	192.168.122.12	4 vCPU	16 GB	60 GB

Cluster installation was automated using Bash scripts included in the repository. The master script performs a hostname preflight check (preventing etcd errors), configures sysctl parameters, installs K3s, and outputs the join token. The worker script validates the hostname format using a regex pattern (k3s-workerN) and verifies master API reachability before joining — eliminating the class of errors that caused POST-001.

Chapter 3: FinOps — Cloud Cost Control

Following SRE best practices, budget and cost alert configuration was completed as the very first action — before any cloud resources were provisioned. This prevents uncontrolled spending, which is critical in SaaS environments managing infrastructure for multiple customers simultaneously.

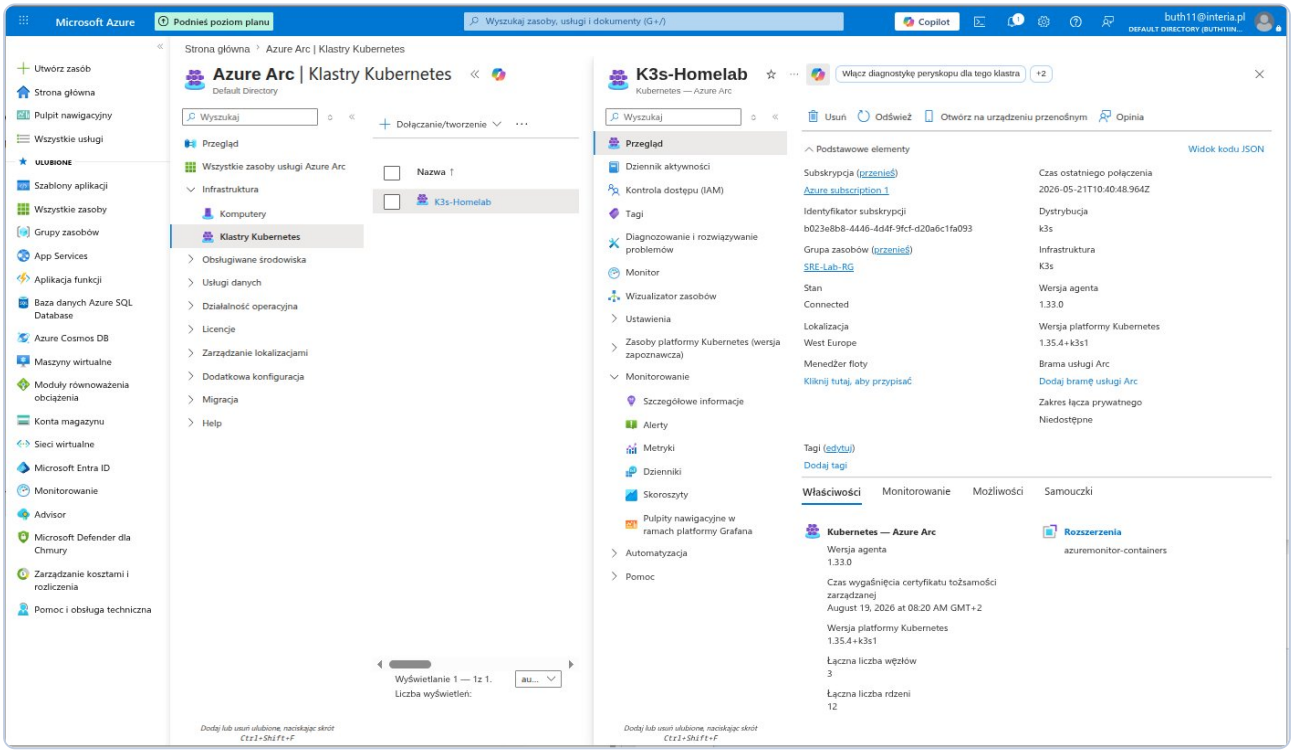


> Screenshot 1: Azure Cost Management — SRE-Lab-Budget (5 USD/month) with email alerts at 50% and 100% thresholds. Final result: 0.00 USD throughout the entire project.

Alert thresholds: email notification at 50% budget consumption (2.50 USD) and 100% (5.00 USD). Zero cloud costs were incurred throughout the entire research period.

Chapter 4: Hybrid Integration — Azure Arc

Azure Arc enables management of the local Kubernetes cluster from the Azure portal. The Arc agent installed on the K3s cluster establishes an outbound connection (HTTPS/443, mTLS) to Azure infrastructure, eliminating the need to open inbound firewall ports.

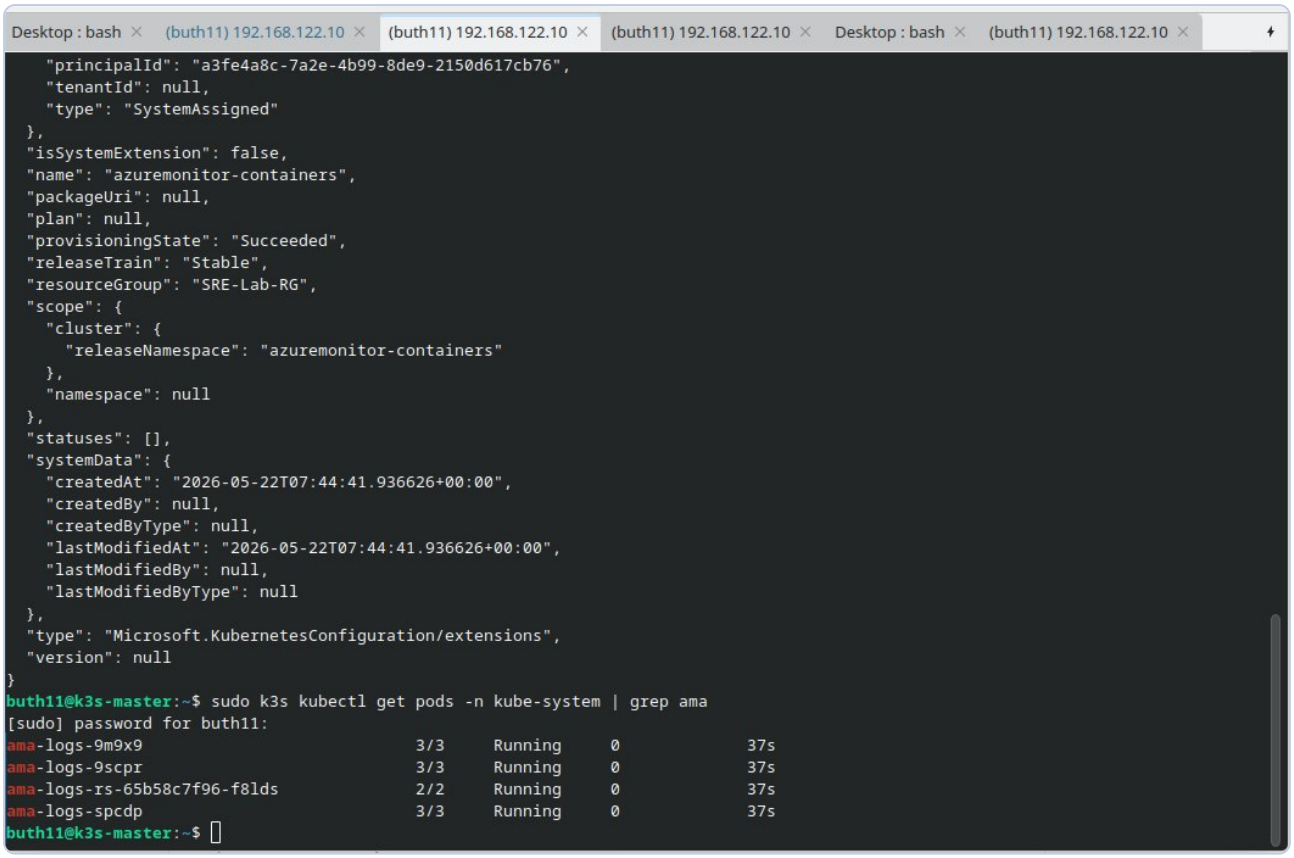


> Screenshot 2: Azure Arc — K3s-Homelab cluster with Connected status. Distribution: k3s v1.35.4+k3s1, 3 nodes, 12 cores, azuremonitor-containers extension active.

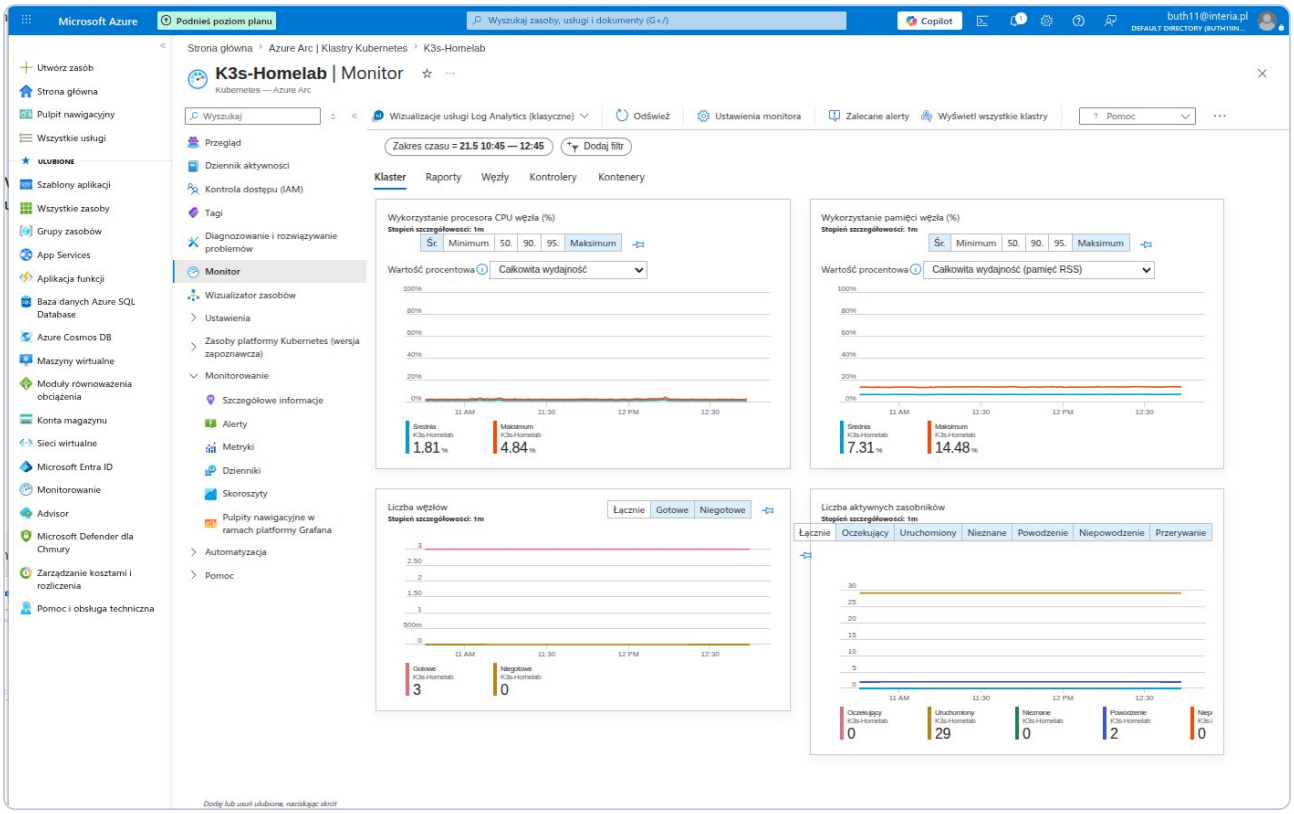
After VM restart, the cluster automatically reconnected to Azure Arc within approximately 3 minutes without any manual intervention — Arc agents are configured as systemd services that start automatically with the OS.

Chapter 5: Observability — Container Insights and Azure Monitor

Container Insights was deployed as an Azure Arc extension using a single CLI command. Four AMA (Azure Monitor Agent) pods were launched on the cluster: three DaemonSet instances (one per node) and one ReplicaSet for aggregate metrics.



> Screenshot 3: AMA pods (ama-logs) in kube-system namespace — 4 agents Running, 0 restarts, 37 seconds after installation.



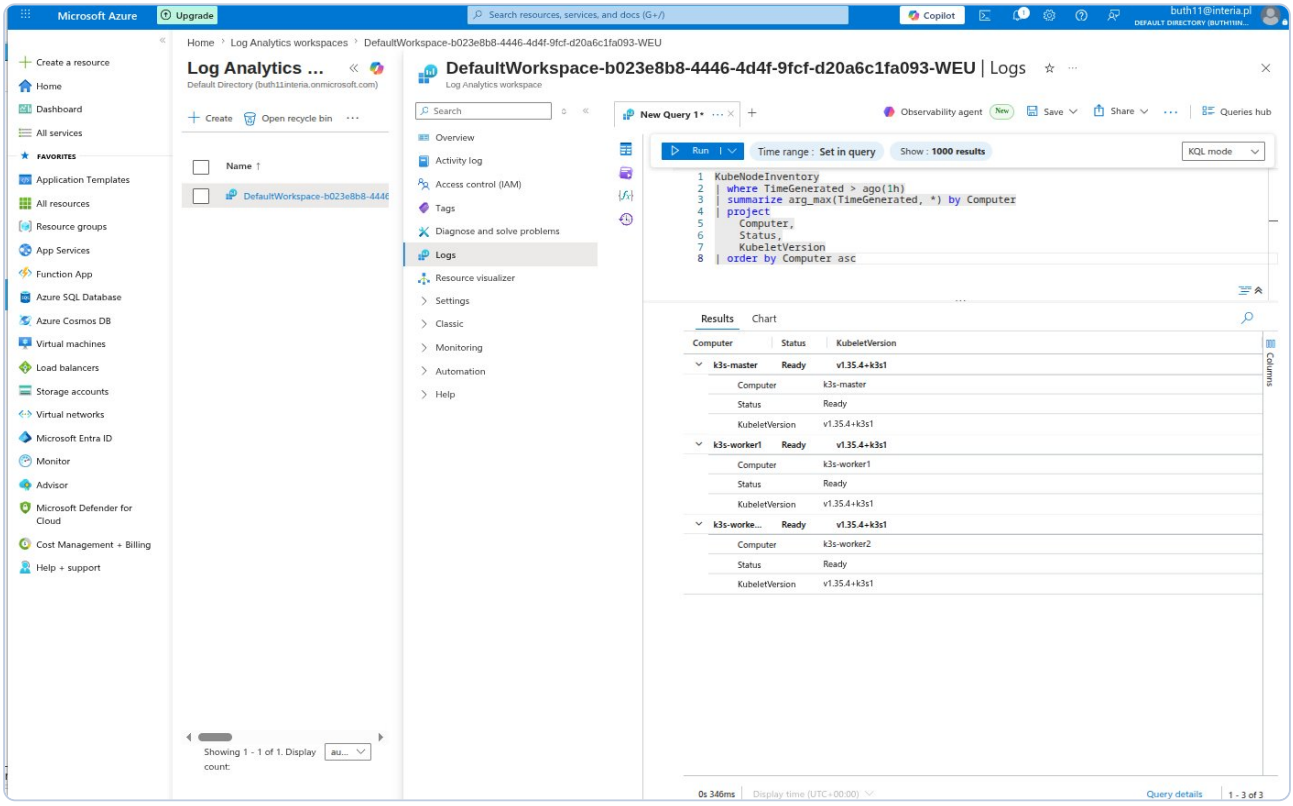
Screenshot 4: Azure Monitor Container Insights — CPU avg 1.58%/max 2.30%, RAM avg 6.27%/max 12.01%, Nodes Ready: 3/3, Active pods: 29.

Chapter 6: KQL Queries — Diagnostics and SLO

KQL (Kusto Query Language) is the primary analytical tool for SRE work in the Azure Monitor ecosystem. The following five diagnostic queries were executed against real cluster data, demonstrating a data-driven approach to monitoring and reliability.

Query 1 — Cluster Node Inventory

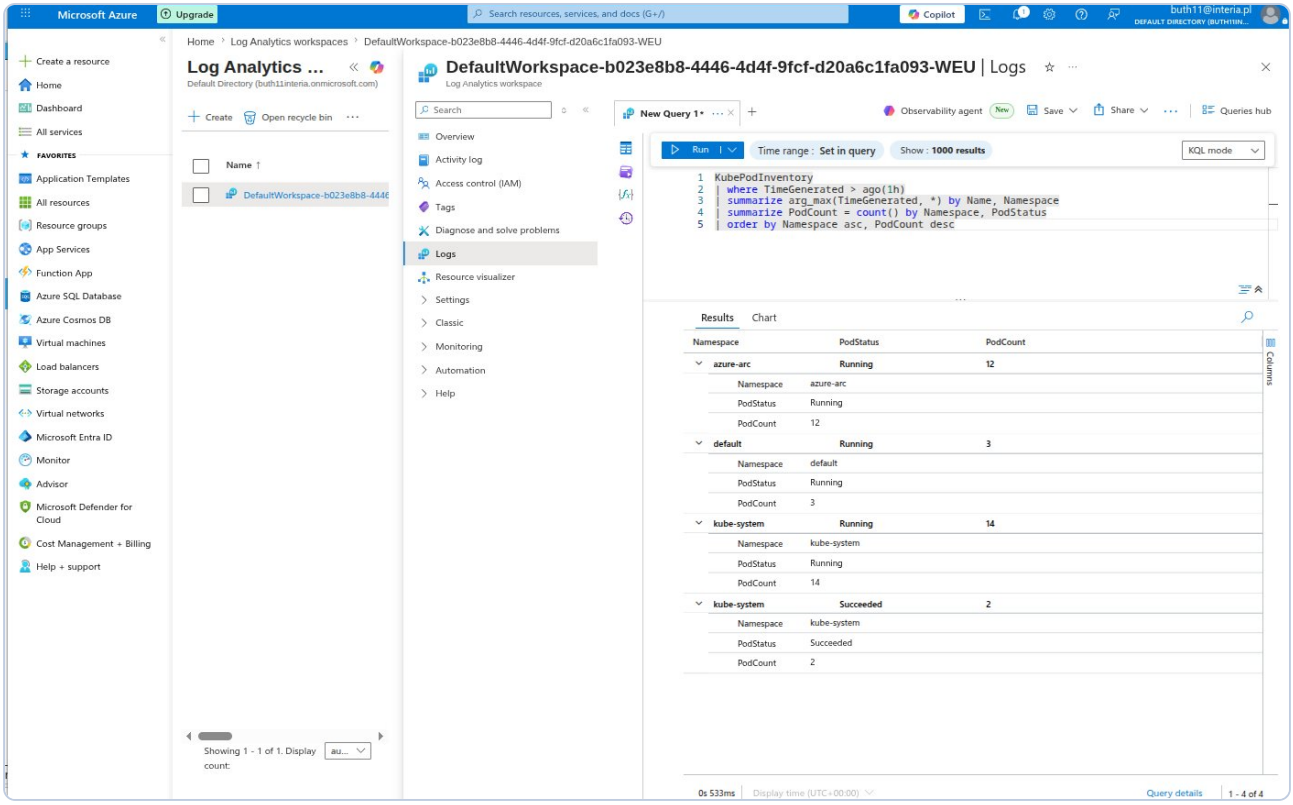
```
KubeNodeInventory
| where TimeGenerated > ago(1h)
| summarize arg_max(TimeGenerated, *) by Computer
| project Computer, Status, KubeletVersion
| order by Computer asc
```



KQL-1: Node inventory — k3s-master, k3s-worker1, k3s-worker2 — all STATUS=Ready, version v1.35.4+k3s1.

Query 2 — Pod Status per Namespace

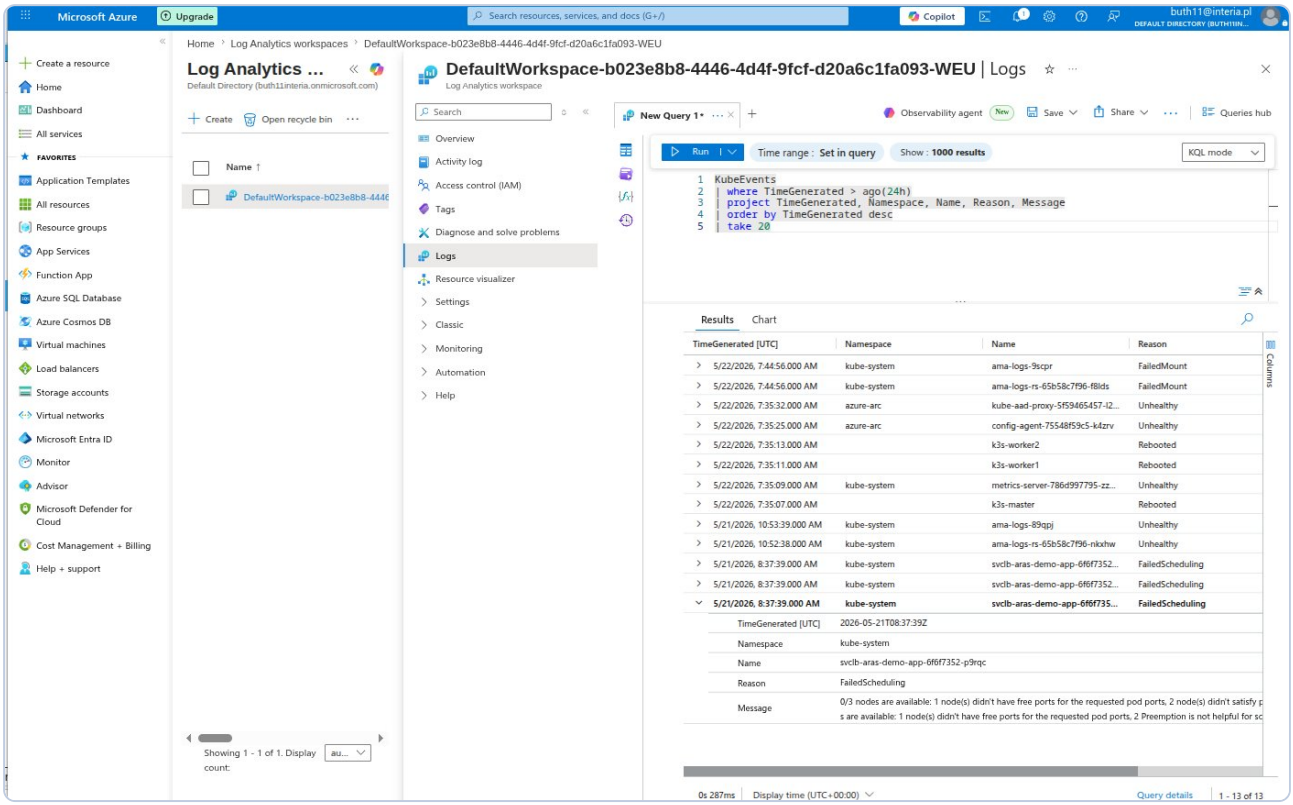
```
KubePodInventory
| where TimeGenerated > ago(1h)
| summarize arg_max(TimeGenerated, *) by Name, Namespace
| summarize PodCount = count() by Namespace, PodStatus
| order by Namespace asc, PodCount desc
```



KQL-2: azure-arc: 12 Running | default: 3 Running (demo app) | kube-system: 14 Running + 2 Succeeded.

Query 3 — Event History (24h)

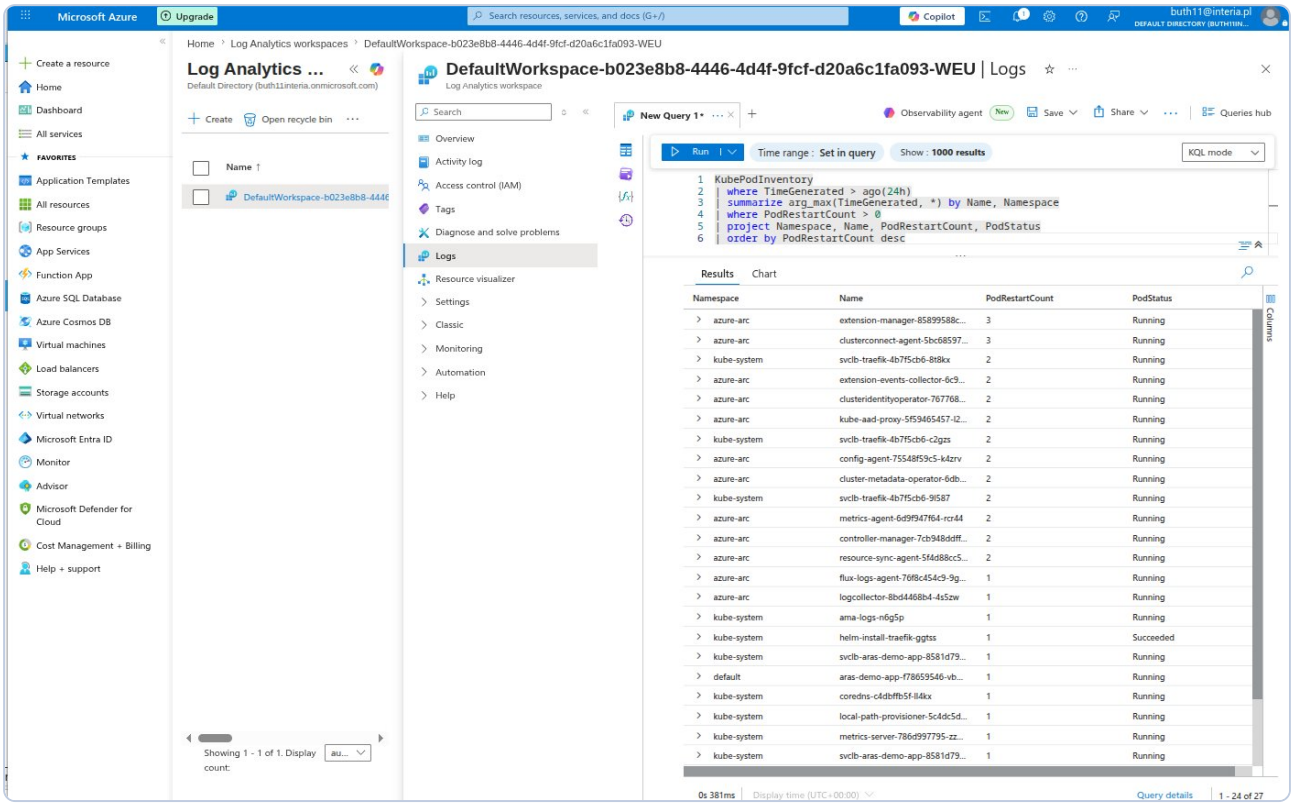
```
KubeEvents
| where TimeGenerated > ago(24h)
| project TimeGenerated, Namespace, Name, Reason, Message
| order by TimeGenerated desc
| take 20
```



KQL-3: 13 events over 24h — Rebooted (VM restart), temporary Unhealthy on Arc agent restart, FailedScheduling on Klipper LB startup. No critical application errors.

Query 4 — Pods with Restarts

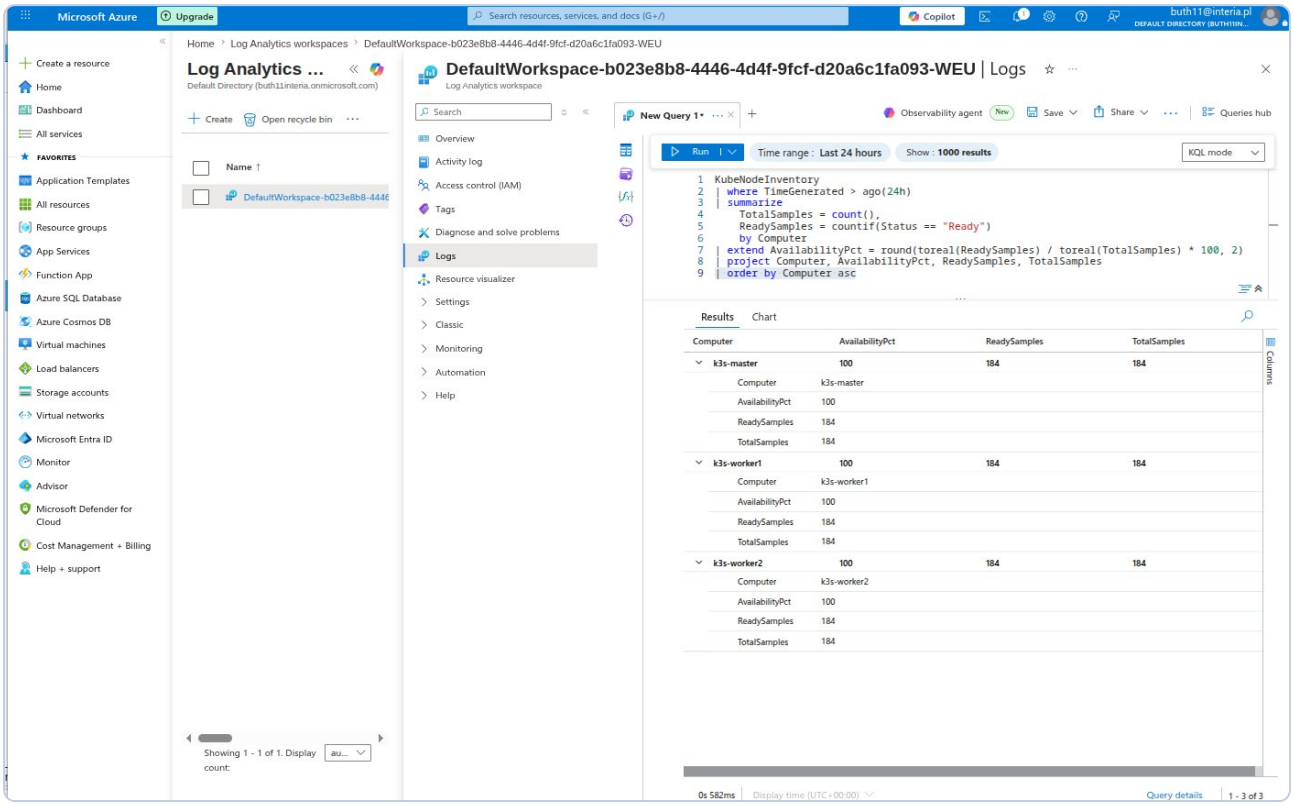
```
KubePodInventory
| where TimeGenerated > ago(24h)
| summarize arg_max(TimeGenerated, *) by Name, Namespace
| where PodRestartCount > 0
| project Namespace, Name, PodRestartCount, PodStatus
| order by PodRestartCount desc
```



KQL-4: 27 pods with restarts after VM reboot — all in Running state. The cluster autonomously restored full capacity without operator intervention.

Query 5 — SLO Availability Calculation (24h window)

```
KubeNodeInventory
| where TimeGenerated > ago(24h)
| summarize
    TotalSamples  = count(),
    ReadySamples  = countif(Status == "Ready")
    by Computer
| extend AvailabilityPct = round(toreal(ReadySamples) / toreal(TotalSamples) * 100, 2)
| project Computer, AvailabilityPct, ReadySamples, TotalSamples
| order by Computer asc
```

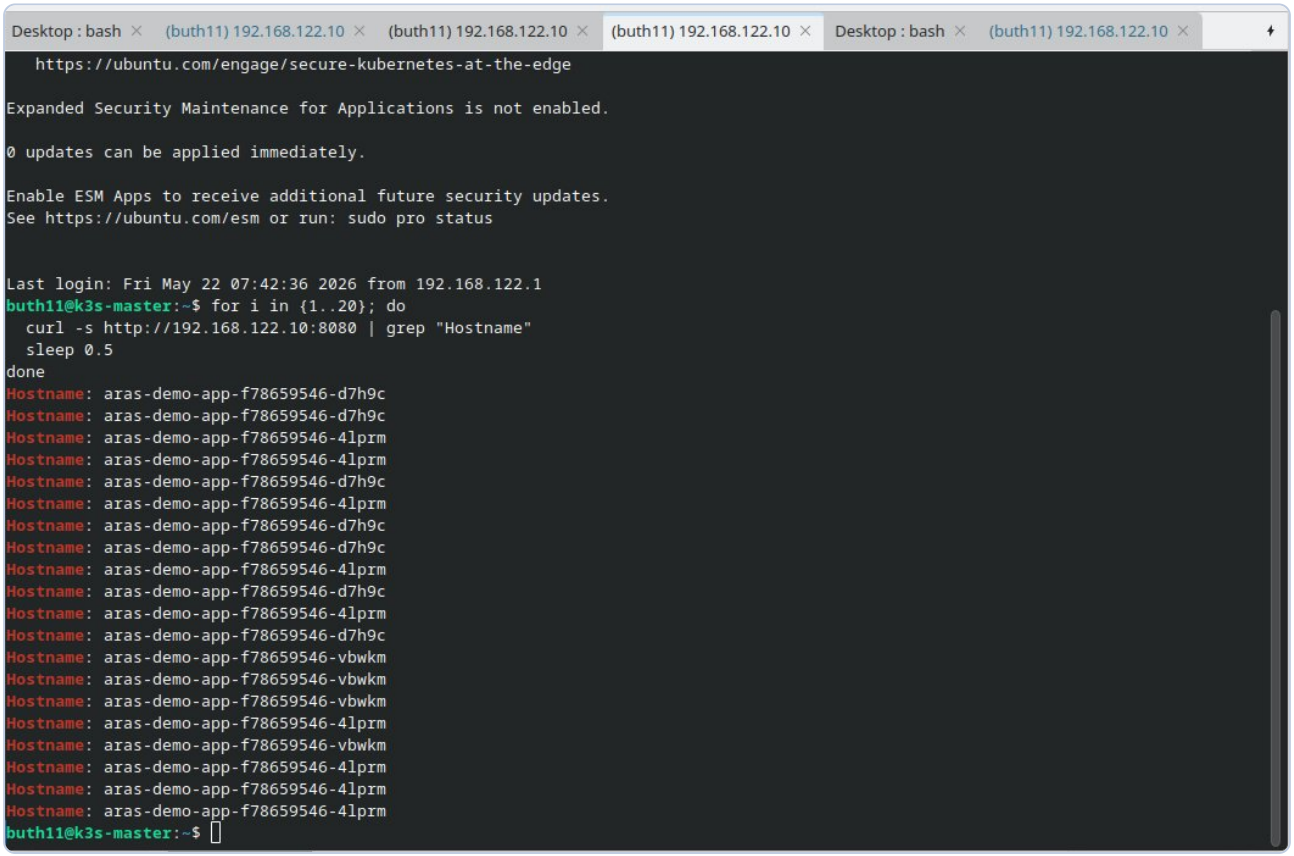


KQL-5: SLO Availability — k3s-master: 100%, k3s-worker1: 100%, k3s-worker2: 100%. All 184/184 samples in Ready state. Error budget intact.

SLO result: 100% node availability across all 3 nodes in a 24-hour window. Enterprise SLO target: 99.9% (43 minutes error budget/month). The cluster consumed zero error budget — even after a full infrastructure restart.

Chapter 7: Load Balancing — Round-Robin Verification

Klipper Load Balancer (built into K3s) distributes traffic across all 3 application replicas via iptables rules. Verification performed by sending 20 sequential HTTP requests and observing hostname values in responses.



> Screenshot 5: Round-robin LB — 20 requests on port 8080 distributed across 3 pods: d7h9c, 4lprm, vbwkm. Concurrent load of 900 requests (3x300 nodes) with zero errors.

Chapter 8: Prometheus and Grafana — Local Monitoring Stack

The local monitoring stack was installed via the `kube-prometheus-stack` Helm chart. A single command deployed: Prometheus, Grafana, Alertmanager, kube-state-metrics, and node-exporter across all nodes.

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.adminPassword=SRE-Lab-2026 \
  --set prometheus.prometheusSpec.retention=24h
```

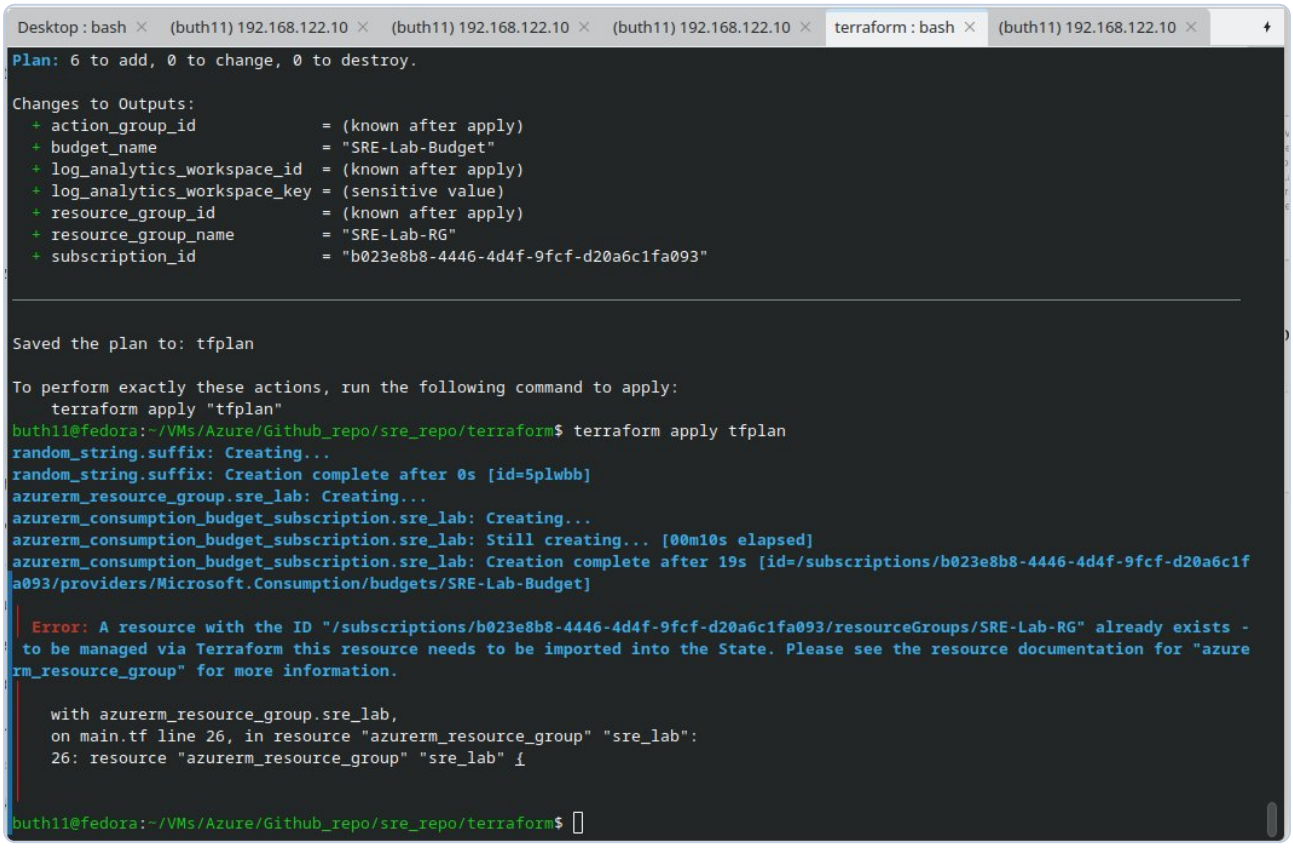
Screenshot 6: Grafana — "Kubernetes / Compute Resources / Cluster" dashboard: CPU Utilisation 2.20%, Memory 12.9%, CPU Quota table per namespace (kube-system: 19 pods, monitoring: 8, default: 3, azure-arc: 12).

Chapter 9: Infrastructure as Code — Terraform

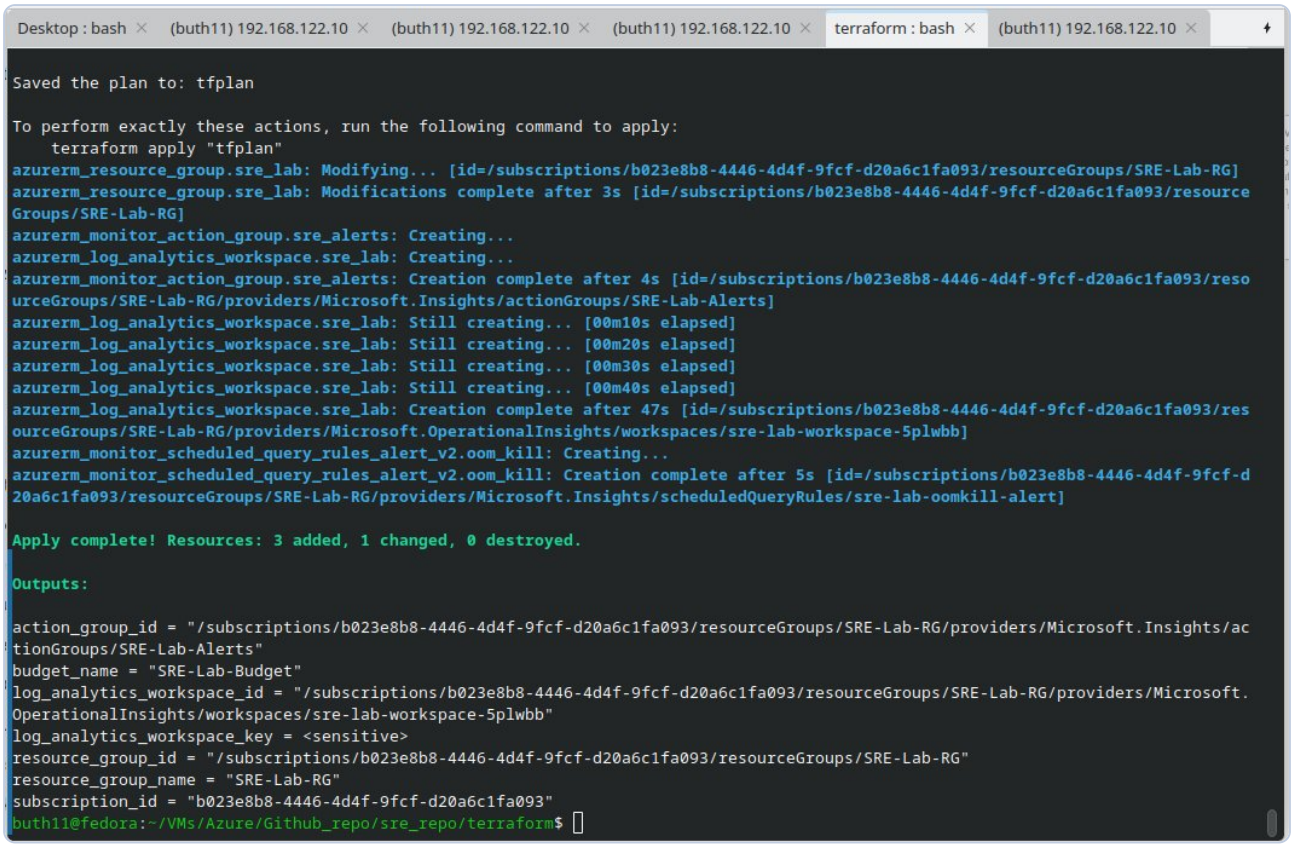
The entire Azure infrastructure is defined in Terraform code, ensuring deployment reproducibility, Git-based change audit trail, and elimination of manual portal errors.

Resources managed by Terraform

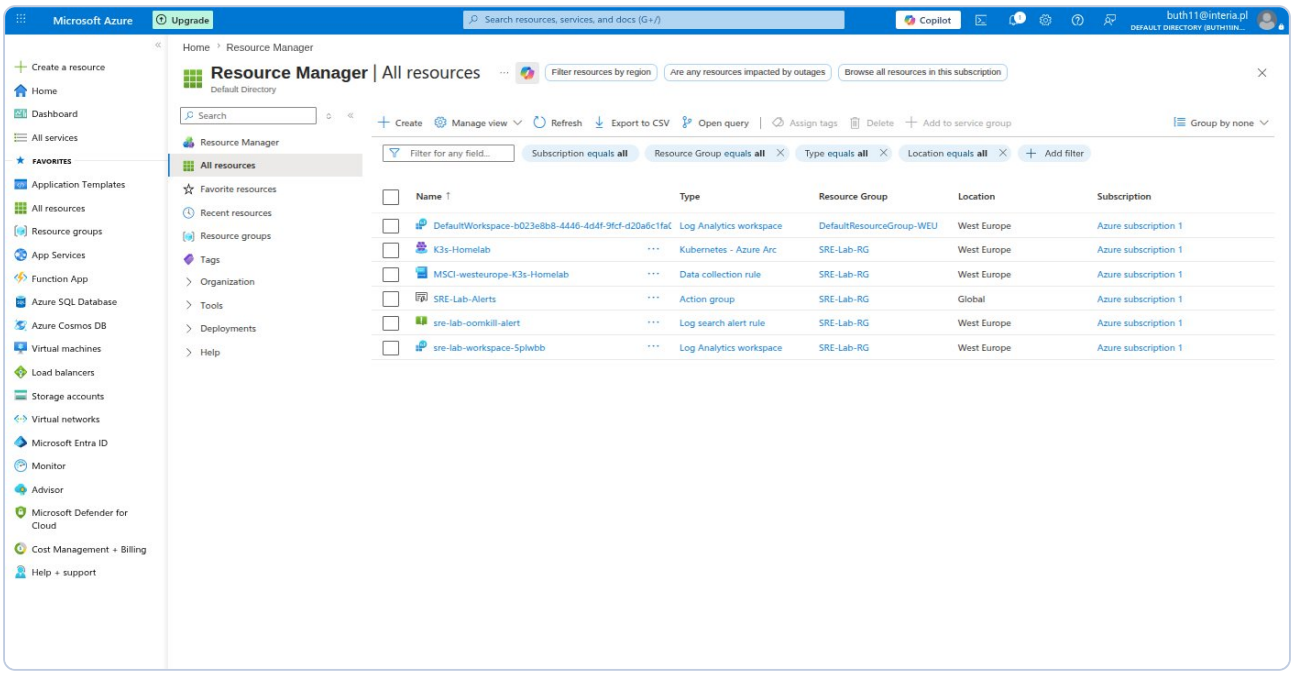
- **azurerm_resource_group** — SRE-Lab-RG in West Europe with tags (managed_by=terraform)
- **azurerm_log_analytics_workspace** — 30-day retention, PerGB2018 SKU
- **azurerm_consumption_budget_subscription** — 5 USD/month with 50% and 100% alerts
- **azurerm_monitor_action_group** — email to buth11@interia.pl
- **azurerm_monitor_scheduled_query_rules_alert_v2** — OOMKill alert using KQL query



Terraform Plan: 6 resources to create, 0 to change, 0 to destroy. Each resource shows detailed configuration including tags: environment=lab, managed_by=terraform, owner=bartosz.suszko.



Terraform Apply: Apply complete! Resources: 3 added, 1 changed, 0 destroyed. Outputs visible: action_group_id, budget_name, log_analytics_workspace_id.



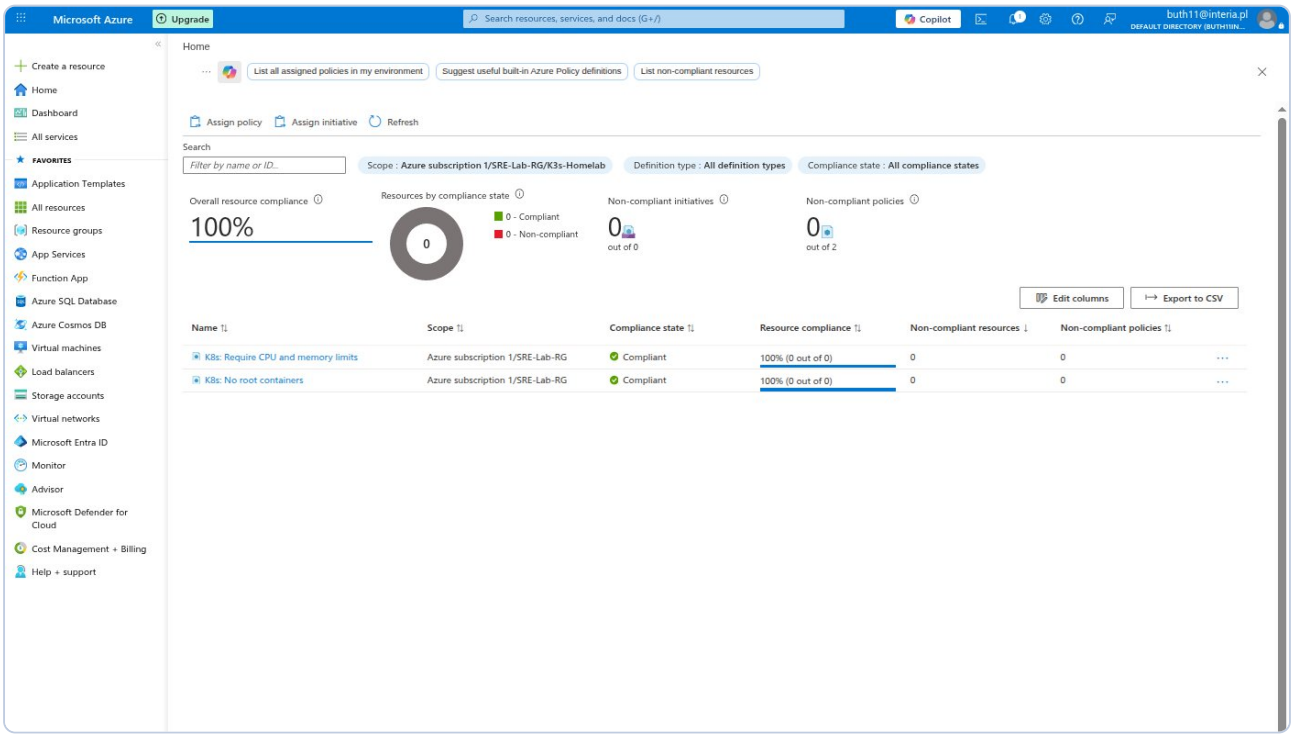
Azure Portal — All Resources: Terraform-managed resources visible in SRE-Lab-RG: SRE-Lab-Alerts (Action group), sre-lab-oomkill-alert (Log search alert rule), sre-lab-workspace-5plwbb (Log Analytics workspace).

Terraform variables include input validation (email regex, budget value range, allowed regions list). Resources that already existed in Azure (Resource Group) were adopted via `terraform import` — this is the standard procedure when introducing IaC into existing infrastructure.

Chapter 9b: Policy as Code — Azure Policy

Azure Policy enables enforcement of security and compliance standards on Kubernetes clusters managed through Arc. Two policies were assigned to the SRE-Lab-RG Resource Group, covering the K3s-Homelab cluster.

Policy	Scope	Parameters	Mode
K8s: No root containers	SRE-Lab-RG	—	DoNotEnforce (Audit)
K8s: Require CPU and memory limits	SRE-Lab-RG	CPU: 500m, RAM: 512Mi	DoNotEnforce (Audit)



> Azure Policy — K3s-Homelab compliance dashboard: Overall compliance 100%, both policies visible (K8s: Require CPU and memory limits, K8s: No root containers), status "Not started" — Policy agent is performing its first cluster assessment.

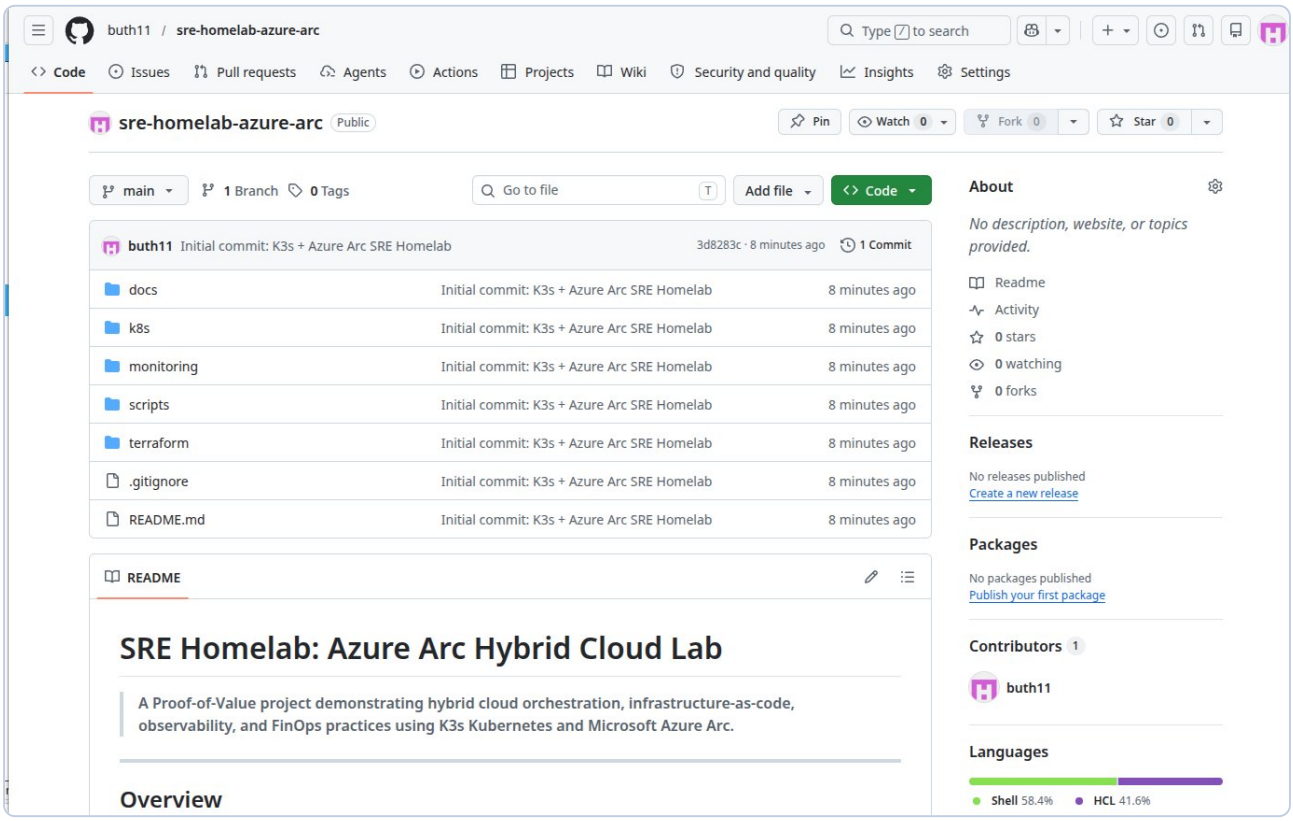
`DoNotEnforce` (Audit) mode allows observation of violations without blocking workloads — this is the recommended approach when first deploying policies to an existing environment. After reviewing results, the mode can be changed to `Deny` for new resources. Policies were assigned via Azure CLI in a single command, enabling management as code (Policy as Code).

All Azure infrastructure is defined in Terraform code, ensuring deployment reproducibility, Git-based change audit trail, and elimination of manual portal errors.

- **azurerm_resource_group** — SRE-Lab-RG in West Europe with consistent tags
- **azurerm_log_analytics_workspace** — 30-day retention, PerGB2018 SKU
- **azurerm_consumption_budget_subscription** — 5 USD/month with 50% and 100% alerts
- **azurerm_monitor_action_group** — email notification to SRE engineer
- **azurerm_monitor_scheduled_query_rules_alert_v2** — OOMKill alert using KQL

Terraform variables include input validation (email regex, budget value range, allowed regions list), preventing misconfiguration at the `terraform validate` stage — before any resources are provisioned.

Chapter 10: GitHub — Documentation and Automation



Screenshot 7: GitHub repo sre-homelab-azure-arc — public, structure: docs/, k8s/, monitoring/, scripts/, terraform/. Languages: Shell 58.4%, HCL 41.6%.

The README.md contains: a Mermaid architecture diagram, technology stack table, real measurement results, and complete step-by-step instructions for reproducing the environment. The repository is publicly accessible for technical review by any interviewer.

Chapter 11: Postmortem — POST-001 etcd Split-Brain

Field	Value
Incident ID	POST-001
Severity	P2 — Degraded cluster capacity
Duration	Under 7 minutes (detection to resolution)
Impact	k3s-worker1 unavailable, cluster at 2/3 capacity
Root Cause	Hostname typo: ks3-worker1 instead of k3s-worker1
Status	Resolved — action items implemented

A typo during node provisioning (ks3 instead of k3s) caused registration of a wrong identity in etcd. Attempting to rename the node with `hostnamectl` triggered etcd's Split-Brain protection mechanism. Key lesson: etcd treats node identity as immutable — the only safe remediation is to delete the stale record via the Kubernetes API and re-register from scratch. Installation scripts were updated with a hostname regex validation as a preflight check, eliminating the possibility of this error class recurring.

Chapter 12: Results and Conclusions

Metric	Value
Cluster nodes	3 / 3 Ready
Running pods	29 (Arc + AMA + K3s + App)
CPU utilisation	avg 2.20% / max 4.84%
RAM utilisation	avg 12.9% / max 14.48%
SLO Availability (24h)	100% — 184/184 samples Ready
Arc agent uptime	12 agents, 0 critical errors
Load balancer	Round-robin across 3 nodes verified
Azure cost	0.00 USD throughout entire project
POST-001 resolution time	Under 7 minutes
IaC files (Terraform)	5 resources with input validation

This project demonstrates the full SRE infrastructure lifecycle: design, deployment, automation, observability, and incident response. Every component of the technology stack required for a Senior SRE role has been installed, configured, and verified against a real hybrid environment — not a simulated or cloud-only setup.